Architecture Design for Turing Test and CAPTCHA

NAME – AYUSHMAN MOHAPATRA

ROLL – SE24UCSE258

BRANCH – CSE (CSE 3)

1. Architecture for Implementing the Turing Test

1.1 INTRO

The Turing Test evaluates whether a machine can exhibit intelligent behaviour indistinguishable from a human through conversation. To implement such a system practically, we must design both:

- **A chat based evaluation platform**
- **An AI conversational agent**
- **A judging mechanism**

The architecture must ensure fairness, anonymity, and realistic human-like interaction.

To pass the test, a system must include:

- **Natural Language Processing (NLP)**
- **Knowledge Representation**
- **Automated Reasoning**
- **Machine Learning**

1.2 Overall System Architecture

The system consists of the following major components:

- **User (Judge) Interface**
- **Conversation Management Server**
- **Identity Router**
- **AI Agent**
- **Human Participant**
- **evaluation module**

1.3 Component-Level Architecture

1. Judge Interface (Frontend)

- **Web based text chat interface.**
- **Allows the human judge to ask questions.**
- **Displays responses from either a human or AI.**
- **At the end of the session, the judge selects whether the participant was human or machine.**

Key Requirements:

- **Clean text only communication (to avoid physical cues).**
- **Session timer.**
- **No identity disclosure**

2. Conversation Management Server

it's the core backend system.

Functions:

- **Creates and manages chat sessions.**
- **Stores conversation logs.**
- **Controls time and message flow.**
- **Ensures anonymity.**
- **Prevents information leakage.**

acts as an intermediary between the judge and the responder.

3. Identity Router (for maintaining fairness)

- **Randomly assigns either a human participant or AI agent to the judge.**
- **Ensures unbiased testing.**
- **Prevents predictable patterns.**

4. AI Agent Architecture

The AI agent must simulate human like intelligence. Internal structure typically includes:

a) Natural Language Processing (NLP) Module

- **Understands input text.**
- **Performs tokenization and semantic analysis.**
- **Detects intent.**

b) Context Manager

- **Maintains conversation history.**
- **Tracks topic continuity.**
- **Remembers previous answers.**

Without this, responses would feel disconnected.

c) Knowledge Base

- **Stores general world knowledge.**
- **May use structured databases or retrieval systems.**
- **Can integrate external APIs.**

d) Reasoning / Response Generator

- **Uses language models or dialogue systems.**
- **Generates coherent and context-aware responses.**

e) Humanization Layer

To improve realism:

- **Adds slight response delay.**
- **Introduces natural conversational tone.**
- **Avoids overly perfect grammar.**
- **Simulates personality traits.**

This layer increases the chances of passing the test.

5. Evaluation Module

After the conversation:

- **The judge makes a decision (Human or AI).**
- **System records accuracy.**
- **Statistical analysis determines if AI performs better than random guessing.**

If judges cannot reliably distinguish the AI from humans, the AI passes the Turing Test.

2. Architecture for Implementing CAPTCHA

2.1 Introduction

CAPTCHA (Completely Automated Public Turing Test to Tell Computers and Humans Apart) is designed to differentiate humans from automated bots.

Unlike the Turing Test, where machines try to appear human, CAPTCHA requires humans to prove they are not machines.

2.2 Overall CAPTCHA System Architecture

The CAPTCHA system consists of:

1. **Challenge Generator**
2. **Challenge Presentation Interface**
3. **User Response Collector**
4. **Verification Engine**
5. **Decision Module**

2.3 Component-Level Architecture

1. Challenge Generator

Generates tasks that are easy for humans but difficult for bots.

Types of CAPTCHA challenges:

a) Text Based CAPTCHA

- **Random string generator.**
- **Font randomization.**
- **Character distortion.**
- **Background noise injection.**

Pipeline:
Random String → Distortion Engine → Noise Injection → CAPTCHA Image

b) Image-Based CAPTCHA

Example:
"Select all images containing traffic lights."

Pipeline:
Image Dataset → Random Category Selection → Grid Formation → Display

Humans use visual recognition skills that are harder for automated scripts.

c) Behavioral CAPTCHA

Instead of puzzles, analyze user interaction:

- **Mouse movement patterns**
- **Click timing**
- **Scroll behavior**
- **Typing speed**

Architecture:
User Interaction Tracker → Feature Extractor → Machine Learning Classifier → Human Probability Score

If probability exceeds threshold → Access Granted.

2. Verification Engine

Responsible for validating responses.

For text/image CAPTCHA:

- **Compare user response with correct answer.**
- **Allow minor tolerance if needed.**

For behavioural CAPTCHA:

- **Feed extracted features into trained classifier.**
- **Generate confidence score.**
- **Apply threshold decision.**

2.4 Security Considerations

- **Prevent replay attacks.**
- **Use time limited tokens.**
- **Regularly update challenge database.**
- **Protect against machine learning-based bot attacks.**

2.4 Security Considerations

- **Prevent replay attacks.**
- **Use time-limited tokens.**
- **Randomize challenges.**
- **Protect against machine learning-based bot attacks**